>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 1 Lecture Notes: Course Introduction, the Concept of an Algorithm, and Problem-Solving Steps

Asst. Prof. Hasan Oğuz, PhD
Istanbul Okan University · Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 1 slides and add the details that do not fit on a slide: full explanations, every worked example with its trace table, a pseudocode reference card, an English–Turkish glossary and graded exercises. Week 1 is *conceptual*: we do not write C# yet. Read Sections 2–4 before Week 2, then do the exercises in Section 6.

Contents

1 About the course **2**
1.1 Aims and learning outcomes . . . 2
1.2 Weekly plan . . . 3
1.3 Assessment and attendance . . . 3
1.4 Software (install before Week 3) . . . 3

2 How a computer solves problems **4**
2.1 Input – Process – Output . . . 4
2.2 Fast, but completely literal . . . 4
2.3 From an idea to a running program . . . 4

3 What is an algorithm? **5**
3.1 The Exact Instructions Challenge . . . 5
3.2 Definition . . . 5
3.3 The five properties of an algorithm . . . 6
3.4 Quiz: algorithm or not? . . . 6
3.5 An everyday algorithm: brewing Turkish tea . . . 6
3.6 Three structures are enough . . . 7
3.7 Four ways to write an algorithm . . . 7
3.8 Pseudocode conventions for MCMP113 . . . 8

4 Problem-solving steps **8**
4.1 The five-step method . . . 8

4.2 Worked example 1: average of three exam grades . . . 9
4.2.1 Step 1 – Understand: the IPO table . . . 9
4.2.2 Step 2 – Design . . . 9
4.2.3 Step 4 – Test: choosing good test cases . . . 9
4.3 Worked example 2: the sum $1 + 2 + \cdots + N$ and trace tables . . . 10
4.4 Worked example 3: guess my number – efficiency . . . 11
4.5 Worked example 4: Euclid's greatest common divisor . . . 11
4.6 Worked example 5: an algorithm in your wallet – T.C. Kimlik No . . . 12
4.7 Common mistakes: bad algorithm, good algorithm . . . 13
4.8 Sneak peek: the same algorithm in C# . . . 13

5 Summary **14**

6 Exercises **14**
6.1 In-class exercises (groups of 2–3) . . . 14
6.2 Homework (due before Week 2) . . . 15
6.3 Extra practice . . . 15

References and further reading **15**

>_ 1 About the course

1.1 Aims and learning outcomes

MCMP113 teaches you to *think like a programmer*: to turn a problem stated in everyday language into a precise sequence of steps, and then into a working C# console application. By the end of the semester you should be able to:

1. **Think algorithmically**: break a real problem into precise, finite, ordered steps.
2. **Draw and read flowcharts**: express an algorithm visually before writing any code.
3. **Write C# console applications** using variables, operators, decisions and loops correctly.
4. **Work with arrays**: store, search, sort and process collections of data.
5. **Structure programs with functions** that take parameters and return values.
6. **Test and debug**: trace code by hand and locate errors systematically.

Outcomes 1–2 do not depend on any programming language. They are the real skill; C# is the vehicle we use to practise it.

1.2 Weekly plan

Week	Topic	Preparation
1	Course introduction · the concept of an algorithm · problem-solving steps	Lecture notes
2	Flowcharts and their symbols · basic flowchart examples	Lecture notes
3	Introduction to programming languages · variables · data types · constants	Notes, sample code
4	Arithmetic, logical and relational operators	Notes, sample code
5	Decision structures: `if-else`, `if-else if`, `switch-case`	Notes, sample code
6	Loops: the `for` loop	Notes, sample code
7	Loops: `while` and `do-while`	Notes, sample code
8	**MIDTERM EXAM**	Review weeks 1–7
9	One-dimensional arrays: declaration, assignment, element access	Notes, sample code
10	Array operations: searching, sorting, finding the maximum/minimum	Notes, sample code
11	Multi-dimensional arrays (matrices)	Notes, sample code
12	Functions: definition, calling, parameter passing	Notes, sample code
13	Value-returning and `void` functions · scope	Notes, sample code
14	Basic file operations · semester review	Lecture notes
15	**FINAL EXAM**	Review all topics

1.3 Assessment and attendance

Component	Week	Weight
Midterm exam	8	40%
Final exam	15	60%

Both exams test *reasoning*: tracing algorithms, drawing flowcharts, and writing short programs by hand. The weekly exercises are not graded, but they are the most direct preparation for the exams. Attendance follows the university regulations.

1.4 Software (install before Week 3)

All tools are free and run on Windows, macOS and Linux.

- **.NET 10 SDK** (the current long-term-support release): `https://dotnet.microsoft.com/en-us/download/dotnet/10.0`
- **Visual Studio Code**: `https://code.visualstudio.com/`
- **C# Dev Kit** extension for VS Code (search "C# Dev Kit" in the Extensions panel).

Check your installation in a terminal (the exact version number may differ):

```
> dotnet --version
10.0.100
> dotnet new console -n Hello
> cd Hello
> dotnet run
Hello, World!
```

Why .NET 10 and not .NET 8?

Support for .NET 8 ends in November 2026, in the middle of our semester; .NET 10 is supported until November 2028. Code written in this course runs on both, but install 10.

>_ 2 How a computer solves problems

2.1 Input – Process – Output

Every program, from a calculator to a video game, follows the same pattern (Figure 1): data comes **in**, the processor **processes** it by following instructions, and results go **out**.

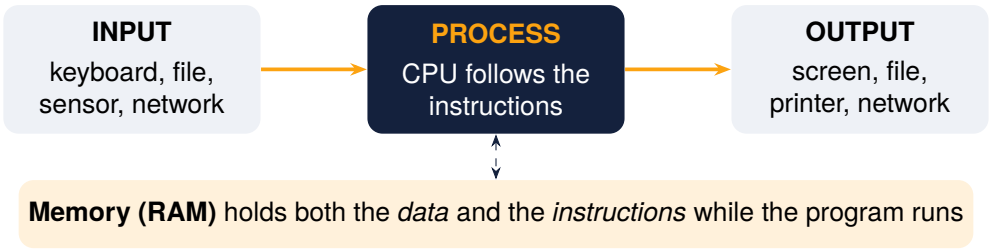


Figure 1: The input–process–output model and the stored-program (von Neumann) idea.

The idea that the program's instructions are stored in the same memory as its data is due to John von Neumann (1945) and is still the architecture of every laptop and phone. RAM is *volatile*: when the power is off it is empty, whereas files on a disk or SSD remain.

2.2 Fast, but completely literal

A single modern processor core executes on the order of 10^9 simple instructions per second. Yet every one of those instructions belongs to only four families:

1. **arithmetic** (add, subtract, multiply, divide),
2. **comparison and logic** (is $a < b$? AND, OR, NOT),
3. **data movement** (copy a value from memory to a register and back),
4. **jumps** (continue at another instruction, possibly only if a condition holds).

The computer never guesses what you *meant*. It does exactly what it is told, in exactly the order it is told. Consequently:

Key insight

The difficult part of programming is not typing code. It is deciding the exact, unambiguous sequence of steps that solves the problem, the **algorithm**. The programming language only writes those steps in a form the machine can execute.

2.3 From an idea to a running program

Figure 2 shows what happens to your work. You write the first two boxes; the tools do the rest when you type `dotnet run`.

The C# compiler (called Roslyn) translates source code into **Intermediate Language** (IL), stored in a `.dll` file. When the program starts, the .NET runtime translates the IL into machine code for

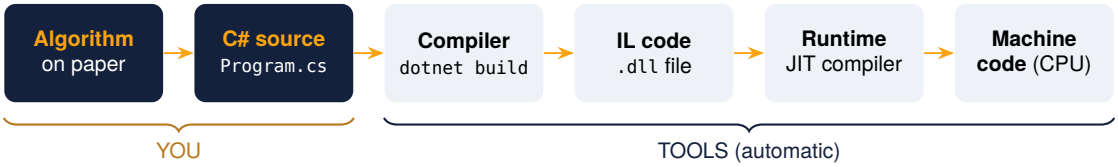


Figure 2: The C# tool chain. IL = Intermediate Language; JIT = just-in-time compilation.

the actual processor (x64, ARM64, ...). This is why the same compiled program runs on Windows, Linux and macOS.

Level	Example: "put 5 in a register, add 3"	Characteristics
Machine code (x86, hex)	`B8 05 00 00 00` `83 C0 03`	The bytes the CPU actually reads. Practically unreadable for humans.
Assembly	`mov eax, 5` `add eax, 3`	One mnemonic per machine instruction; tied to one processor family.
High-level (C#)	`int sum = 5 + 3;`	Close to human thinking; portable across systems.

Table 1: The same computation at three levels of abstraction.

Think about it

For each of the following, name the input, the process and the output: a pocket calculator; an ATM withdrawal; a navigation app computing a route; a thermostat.

>_ 3 What is an algorithm?

3.1 The Exact Instructions Challenge

In class we run a live demonstration in which the class dictates instructions for making a sandwich and a volunteer follows them *literally*: "put the peanut butter on the bread" results in the closed jar being placed on the loaf. While observing, note down (i) which instruction was ambiguous, (ii) which step was missing, (iii) which step was in the wrong order, (iv) whether the "algorithm" ever ended, and (v) how the class fixed it. The volunteer behaves exactly like a processor, and the class's cycle of *write, test, fail, fix* is exactly how programs are developed.

3.2 Definition

Definition – Algorithm

An **algorithm** is a **finite** sequence of **well-defined** steps that transforms a given **input** into the desired **output**, and then stops.

Where the word comes from. Muhammad ibn Mūsā al-Khwārizmī (c. 780–850) worked at the House of Wisdom in Baghdad. His book on calculating with Hindu–Arabic numerals introduced the decimal system to Europe, where his name was Latinised as *Algoritmi*; hence *algorithm*. The title of his other book, *al-Jabr*, gave us the word *algebra*.

Algorithm versus program. An algorithm is an *idea*, independent of any language; it can be written in words, in pseudocode or as a flowchart. A **program** is one *implementation* of that idea in a particular programming language. One algorithm can be implemented in C#, Python, Java or any other language: the relation is one-to-many. A recipe is an algorithm; the dish cooked in a particular kitchen is a program run.

3.3 The five properties of an algorithm

Following D. E. Knuth, a procedure qualifies as an algorithm only if it has all five properties below.

Property	Meaning	*Violated by …*
Finiteness	It stops after a finite number of steps.	*"Keep stirring." (until when?)*
Definiteness	Every step is precise and unambiguous.	*"Add some sugar." (how much?)*
Input	It has zero or more clearly specified inputs.	*"Take the bread." (which bread?)*
Output	It produces at least one result related to the input.	*An empty plate.*
Effectiveness	Every step is basic enough to be carried out exactly.	*"Make it tasty."*

For practical programming we add a sixth requirement: **correctness**. The algorithm must produce the right output for *every* valid input, including unusual ones (the *edge cases*).

3.4 Quiz: algorithm or not?

Decide for each item whether it is an algorithm. If not, name the property that fails.

1. The instructions on a shampoo bottle: "Lather. Rinse. Repeat."
2. The long-division method you learned in primary school.
3. "Print the largest prime number."
4. A recipe step: "Add salt to taste."
5. Finding a word in a paper dictionary by opening it near the word's first letter and moving forwards or backwards.
6. "Choose the best football team in the world."

Answers

(1) No: it never stops (finiteness). (2) Yes. (3) No: Euclid proved that there are infinitely many primes, so no largest one exists; the procedure never finishes and produces no output. (4) No: "to taste" is not precise (definiteness). (5) Yes: the dictionary is finite and alphabetically ordered, so the search always ends. This is the idea behind *binary search* (Week 10). (6) No: there is no agreed criterion (definiteness, effectiveness).

3.5 An everyday algorithm: brewing Turkish tea

Compare the vague version "boil water, add tea, serve" with the precise version below. The colours mark the three kinds of structure discussed in the next subsection.

Algorithm 1 – Brewing Turkish tea

1. Fill the kettle (bottom pot) with 1 L of water.
2. Put 4 teaspoons of black tea into the teapot (top pot).
3. Place the teapot on the kettle and turn the stove on.
4. **WHILE** the water is not boiling: wait 1 minute.
5. Pour boiling water into the teapot until it is $\frac{3}{4}$ full.
6. Lower the heat and wait 15 minutes.
7. **IF** the guest wants strong tea **THEN** fill the glass $\frac{2}{3}$ with tea, **ELSE** fill it $\frac{1}{3}$ with tea.
8. Top up the glass with hot water. **STOP**.

3.6 Three structures are enough

Every algorithm can be built from only three control structures. This is the **structured program theorem** of Böhm and Jacopini (1966).

Structure	Meaning	Pseudocode example	C# in
Sequence	Steps executed one after another, top to bottom.	**READ** a, b $s \leftarrow a + b$ **PRINT** s	Weeks 3–4
Selection	Choose between paths according to a condition.	**IF** $age \geq 18$ **THEN** **PRINT** `"adult"` **ELSE PRINT** `"minor"`	Week 5
Repetition	Repeat a block while a condition holds.	**WHILE** $i \leq 10$ **PRINT** i; $i \leftarrow i + 1$	Weeks 6–7

This theorem is why the course is ordered the way it is: after sequence, selection and repetition you have, in principle, everything needed to express any computation.

3.7 Four ways to write an algorithm

The same algorithm, "read a number and print its absolute value", can be written in four ways:

(a) Natural language. *Read the number. If it is negative, remove its minus sign. Show the number.*

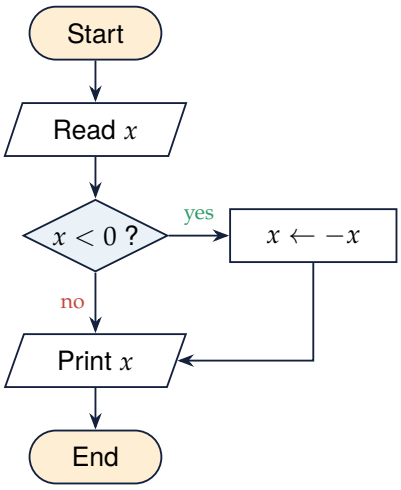
Easy for anyone to read, but often ambiguous.

(b) Pseudocode.

```
1 START
2 READ x
3 IF x < 0 THEN
4     x ← −x
5 END IF
6 PRINT x
7 END
```

Precise and needs no tools; our main notation on paper.

(c) Flowchart (Week 2).



(d) C# code (Week 3 onwards). Executable, but with strict syntax:

```
int x = int.Parse(Console.ReadLine());
if (x < 0)
    x = -x;
Console.WriteLine(x);
```

3.8 Pseudocode conventions for MCMP113

There is no single standard pseudocode. In this course, in all slides, homework and exams, we use the following conventions.

Keyword	Meaning	Example
`START … END`	beginning and end of the algorithm	
`READ x`	input: get a value from the user	**READ** *age*
`PRINT …`	output: show a value or a message	**PRINT** `"Total:"`, *sum*
`x` ← `expression`	assignment: evaluate the right-hand side, store it in x	$total \leftarrow total + price$
`IF c THEN … ELSE … END IF`	selection	**IF** $age \geq 18$ **THEN** …
`WHILE c … END WHILE`	repeat while condition c is true	**WHILE** $i \leq N$
`FOR i` ← `a TO b … END FOR`	repeat for $i = a, a+1, \ldots, b$	**FOR** $i \leftarrow 1$ **TO** 10
`+` `-` `*` `/` `mod`	arithmetic; `mod` is the remainder	$17 \bmod 5 = 2$
`=` $\neq$ `<` $\leq$ `>` $\geq$	comparison (result: true or false)	**IF** $x \neq 0$ **THEN** …
`AND` `OR` `NOT`	combine conditions	**IF** $a > 0$ **AND** $b > 0$

The number-one beginner trap: assignment is not equality

In pseudocode, ← *assigns* and $=$ *compares*. In C#, `=` assigns and `==` compares. The statement $x \leftarrow x + 1$ is not an equation (as an equation, $x = x + 1$ has no solution). It means "the new value of x is the old value of x plus one".

>_ 4 Problem-solving steps

4.1 The five-step method

Inspired by G. Pólya's *How to Solve It* (1945), we solve every programming problem in five steps:

1. **Understand (analyse).** What is given (input)? What is asked (output)? What are the constraints? Which unusual inputs can occur?
2. **Design.** Write the algorithm: numbered steps, then pseudocode and/or a flowchart.
3. **Implement.** Translate the algorithm into C#.
4. **Test and debug.** Run the program (or trace the algorithm by hand) with normal, boundary and invalid inputs.
5. **Document.** Comments, meaningful names, notes for other people and for your future self.

The method is a *cycle*, not a straight line: a failed test sends you back to Design, or even to Understand. Beginners tend to spend most of their time in step 3; experienced programmers spend most of it in steps 1–2, which is why their step 4 is short.

Rule of thumb

Do not start coding until you can explain your algorithm in your own words to a classmate.

4.2 Worked example 1: average of three exam grades

Problem. A student enters three exam grades (0–100). Show the average and whether the student passed (average at least 50).

Step 1 – Understand: the IPO table

Input	Process	Output
g_1, g_2, g_3 (numbers, 0–100)	$avg \leftarrow (g_1 + g_2 + g_3)/3$; compare avg with 50	avg; `"PASSED"` or `"FAILED"`

Questions to ask *before* designing:

- Can a grade be 72.5, or only whole numbers? (This decides the data type in Week 3.)
- What if the user types 120 or -5? (Needs a range check, i.e. selection, Week 5.)
- Is exactly 50 a pass? (Decides between $\geq$ and $>$.)
- Do all exams have equal weight? (If not, a weighted average is needed.)

Step 2 – Design

In plain language: (1) start; (2) read the three grades; (3) add them and divide by 3; (4) show the average; (5) if the average is at least 50 show "PASSED", otherwise "FAILED"; (6) stop. In pseudocode:

Algorithm 2 – Average of three grades

```
START
READ g1, g2, g3
avg ← (g1 + g2 + g3)/3
PRINT avg
IF avg ≥ 50 THEN
    PRINT "PASSED"
ELSE
    PRINT "FAILED"
END IF
END
```

Parentheses matter

Writing $g_1 + g_2 + g_3/3$ is a different algorithm: division is performed before addition, so it computes $g_1 + g_2 + \frac{g_3}{3}$. For $60, 70, 80$ the correct average is 70, whereas the wrong formula gives 156.67. Operator precedence is studied in Week 4.

Step 4 – Test: choosing good test cases

Testing with a single "nice" input proves very little. Choose tests that are most likely to break the algorithm:

Kind of test	g_1, g_2, g_3	Expected result
Normal	60, 70, 80	70.00 – PASSED
Boundary	50, 50, 50	50.00 – PASSED
Just below	49, 50, 50	49.67 – FAILED $(149/3 = 49.\overline{6})$
Maximum	100, 100, 100	100.00 – PASSED
Minimum	0, 0, 0	0.00 – FAILED
Out of range	120, 70, 80	should be rejected, but Algorithm 2 prints 90.00
Wrong type	`abc`, 70, 80	a C# program would crash (`FormatException`)

Boundary tests catch the most errors (for example, writing $avg > 50$ instead of $avg \geq 50$). The last two rows reveal *missing requirements*: Algorithm 2 does not validate its input. As Dijkstra put it: *"Program testing can be used to show the presence of bugs, but never to show their absence."*

4.3 Worked example 2: the sum $1 + 2 + \cdots + N$ and trace tables

Algorithm 3 – Sum of the first N positive integers

```
1  START
2  READ N
3  sum ← 0
4  i ← 1
5  WHILE i ≤ N
6      sum ← sum + i
7      i ← i + 1
8  END WHILE
9  PRINT sum
10 END
```

A **trace table** (also called a *desk check*) executes the algorithm by hand: one column per variable, one row per pass through the loop. For $N = 5$:

pass	i (before test)	$i \leq N$?	sum (after pass)
start	1	–	0
1	1	yes	1
2	2	yes	3
3	3	yes	6
4	4	yes	10
5	5	yes	15
exit	6	no	**15**

Checks. *Termination:* i increases by one in every pass, so it eventually exceeds N. *Edge cases:* for $N = 0$ the loop body never runs and the output is 0, which is correct; for negative N the output is also 0 – is that what the user expects?

A different algorithm for the same problem. The young Gauss is said to have noticed that pairing the first and last terms gives $N + 1$ each time:

$$S = \sum_{i=1}^{N} i, \qquad 2S = \underbrace{(1+N) + (2+(N-1)) + \cdots + (N+1)}_{N \text{ pairs, each equal to } N+1} = N(N+1) \quad \Longrightarrow \quad S = \frac{N(N+1)}{2}. \tag{1}$$

For $N = 5$: $S = 5 \cdot 6/2 = 15$. Algorithm 3 needs N loop passes, the formula needs one multiplication and one division, whatever N is. For $N = 10^9$ the difference is dramatic. *Choosing the better algorithm is a design decision, not a coding one.*

4.4 Worked example 3: guess my number – efficiency

I think of an integer between 1 and 1000; you guess, and I answer "higher", "lower" or "correct".

- **Strategy A (linear):** guess $1, 2, 3, \ldots$. Worst case: 1000 guesses.
- **Strategy B (halving):** always guess the middle of the remaining range. Every answer halves the range, so after k guesses at most $\lfloor 1000/2^k \rfloor$ candidates remain. The worst case is

$$k_{\max} = \lceil \log_2(N+1) \rceil, \qquad N = 1000 \;\Rightarrow\; k_{\max} = 10, \quad \text{since } 2^{10} = 1024 \geq 1001. \tag{2}$$

Algorithm 4 – Guessing by halving

```
START
low ← 1; high ← 1000; found ← false
WHILE NOT found
    guess ← ⌊(low + high)/2⌋
    PRINT guess; READ answer
    IF answer = "correct" THEN
        found ← true
    ELSE
        IF answer = "higher" THEN
            low ← guess + 1
        ELSE
            high ← guess − 1
        END IF
    END IF
END WHILE
END
```

Trace for the secret number 737 (10 guesses):

guess no.	1	2	3	4	5	6	7	8	9	10
guess	500	750	625	687	718	734	742	738	736	**737**
answer	↑	↓	↑	↑	↑	↑	↓	↓	↑	✓

Range size N	100	1 000	10^6	10^9
Strategy A, worst case	100	1 000	1 000 000	1 000 000 000
Strategy B, worst case	7	10	20	30

Strategy A takes time proportional to N (we write $O(N)$); strategy B takes time proportional to $\log_2 N$ ($O(\log N)$). Strategy B is **binary search**, which we implement on arrays in Week 10.

4.5 Worked example 4: Euclid's greatest common divisor

Written down around 300 BC in Euclid's *Elements*, this is one of the oldest non-trivial algorithms still in use; it is part of the arithmetic behind public-key cryptography (RSA) used in secure HTTPS connections.

Algorithm 5 – Euclid's GCD

```
START
READ a, b                                   ▷ positive integers
WHILE b ≠ 0
    r ← a mod b
    a ← b
    b ← r
END WHILE
PRINT a
END
```

a	b	$r = a \bmod b$
48	18	12
18	12	6
12	6	0
6	0	stop: $\gcd(48,18) = 6$

Why it is correct. Any common divisor of a and b also divides $r = a - qb$, and any common divisor of b and r also divides $a = qb + r$; hence $\gcd(a,b) = \gcd(b,\, a \bmod b)$. **Why it terminates.** Since $0 \le r < b$, the value of b strictly decreases in every pass and, being a non-negative integer, must reach 0. If the input has $a < b$, the first pass simply swaps them: for $(18, 48)$ we get $r = 18$, $a = 48$, $b = 18$.

4.6 Worked example 5: an algorithm in your wallet – T.C. Kimlik No

The 11-digit Turkish identity number $d_1 d_2 \ldots d_{11}$ contains two *check digits*. The rules are:

$$d_1 \neq 0, \tag{3}$$

$$d_{10} = \Big(7\,(d_1 + d_3 + d_5 + d_7 + d_9) - (d_2 + d_4 + d_6 + d_8)\Big) \bmod 10, \tag{4}$$

$$d_{11} = \Big(d_1 + d_2 + \cdots + d_{10}\Big) \bmod 10. \tag{5}$$

Worked example with the *made-up* first nine digits 1 2 3 4 5 6 7 8 9: the odd-position sum is 25, the even-position sum is 20, so $d_{10} = (175 - 20) \bmod 10 = 155 \bmod 10 = 5$, and $d_{11} = (45 + 5) \bmod 10 = 0$. The full number is 12345678950. It passes the check but belongs to nobody.

Why it is useful. If exactly one of $d_1, \ldots, d_{10}$ is mistyped, the sum $d_1 + \cdots + d_{10}$ changes by a non-zero amount between -9 and $+9$, so its remainder modulo 10 no longer equals d_{11}. The error is detected instantly, without contacting any database. The same idea protects IBANs (mod 97), EAN-13 product barcodes and credit-card numbers (the Luhn algorithm).

Looking ahead to Week 4

The expression $7 \cdot (\text{odd sum}) - (\text{even sum})$ can be negative (e.g. $7 \cdot 1 - 36 = -29$). Mathematically $-29 \bmod 10 = 1$, but the C# operator `%` gives $-29 \,\%\, 10 = -9$. A correct C# version must use `((x % 10) + 10) % 10`.

4.7 Common mistakes: bad algorithm, good algorithm

Problem	Bad	Good
Ambiguous step	"Divide the numbers."	$avg \leftarrow (g_1 + g_2 + g_3)/3$
No termination	"Keep asking for a password."	Ask at most 3 times, then lock the account.
Missing edge case	$result \leftarrow a/b$	**IF** $b = 0$ **THEN PRINT** `"error"` **ELSE** $result \leftarrow a/b$
Wrong order	**PRINT** sum, then $sum \leftarrow a + b$	$sum \leftarrow a + b$, then **PRINT** sum
Undefined input	"Read the grade."	Read a grade in $[0, 100]$; reject other values.

Each row violates one property: definiteness, finiteness, correctness, sequence and input specification, respectively.

4.8 Sneak peek: the same algorithm in C#

In Week 3 you will write Algorithm 2 as follows. You do not need to understand the syntax yet; notice only that every pseudocode line maps to one or two lines of code.

```
Console.Write("Grade 1: ");
double g1 = double.Parse(Console.ReadLine());
Console.Write("Grade 2: ");
double g2 = double.Parse(Console.ReadLine());
Console.Write("Grade 3: ");
double g3 = double.Parse(Console.ReadLine());

double avg = (g1 + g2 + g3) / 3;          // parentheses!
Console.WriteLine($"Average: {avg:F2}");

if (avg >= 50)
    Console.WriteLine("PASSED");
else
    Console.WriteLine("FAILED");
```

Pseudocode	C#
READ x	`Console.ReadLine()` (plus a conversion such as `double.Parse`)
PRINT x	`Console.WriteLine(x)`
$x \leftarrow e$	`x = e;`
IF c **THEN** … **ELSE** …	`if (c) { …} else { …}`

Turkish-locale computers

On a computer set to Turkish regional settings, `double.Parse` expects a decimal *comma* (72,5), not a point. We will handle this properly in Week 3.

>_ 5 Summary

1. An **algorithm** is a finite, precise recipe that turns input into output and stops.
2. Its five properties are finiteness, definiteness, input, output and effectiveness; in practice we also require correctness.
3. Every algorithm is built from **sequence**, **selection** and **repetition**.
4. Algorithms can be written in natural language, pseudocode, flowcharts or code.
5. Solve problems in five steps: understand, design, implement, test, document, and loop back when a test fails.
6. Trace tables and well-chosen test cases (normal, boundary, invalid) find errors before the computer does.
7. Different algorithms for the same problem can differ enormously in efficiency (sum formula; binary search).

Glossary (English – Turkish)

algorithm	algoritma	input / output	girdi / çıktı
step	adım	assignment	atama
pseudocode	sözde kod	condition	koşul
flowchart	akış şeması	selection (decision)	seçim (karar)
sequence	sıralı yapı	repetition (loop)	tekrar (döngü)
variable	değişken	trace table	izleme tablosu
finiteness	sonluluk	edge / boundary case	sınır durumu
definiteness	belirlilik / kesinlik	remainder (mod)	kalan (mod)
effectiveness	etkinlik / yapılabilirlik	greatest common divisor	en büyük ortak bölen (EBOB)
compiler	derleyici	check digit	kontrol basamağı
source code	kaynak kod	efficiency	verimlilik
bug / debugging	hata / hata ayıklama	binary search	ikili arama

>_ 6 Exercises

For every exercise: write the IPO table, the numbered steps and the pseudocode (using the conventions of Section 3.8), then trace your algorithm with at least one normal and one edge-case input. Difficulty: ⋆ basic, ⋆⋆ intermediate, ⋆⋆⋆ challenging.

6.1 In-class exercises (groups of 2–3)

A. ⋆ **Even or odd.** Read an integer; print whether it is even or odd.

B. ⋆⋆ **Leap year.** Read a year; it is a leap year if it is divisible by 4 and not by 100, or if it is divisible by 400.

C. ⋆⋆ **ATM withdrawal.** Given the account balance and a requested amount, allow the withdrawal only if the balance is sufficient and the amount is a positive multiple of 10; print the new balance or a reason for refusal.

D. ⋆ **Countdown.** Read N; print $N, N-1, \ldots, 1$ and then "Go!".

E. ⋆⋆ **Guess the number.** The computer holds a secret number; the user guesses repeatedly and receives "higher" or "lower" until correct. Also count the guesses.

F. ⋆ **Temperature conversion.** Read a temperature C in degrees Celsius and print it in Fahrenheit, $F = \frac{9}{5}C + 32$.

6.2 Homework (due before Week 2)

H1. Complete the two in-class exercises that your group did not do in class.

H2. ⋆ Write an algorithm for a daily task of your choice that contains at least one **IF** and one **WHILE**.

H3. ⋆ Trace Euclid's algorithm (Algorithm 5) for $(84, 36)$ in a table.

H4. Install the .NET 10 SDK, VS Code and the C# Dev Kit; run the four commands of Section 1.4 and post a screenshot of the output to the course group.

6.3 Extra practice

P1. ⋆ Trace Algorithm 3 for $N = 3$ and for $N = 0$. How many times is the loop condition tested in each case?

P2. ⋆ Use Gauss's formula to compute $1 + 2 + \cdots + 100$ and $1 + 2 + \cdots + 1000$.

P3. ⋆⋆ What is the worst-case number of guesses with the halving strategy for a secret number between 1 and 500? Between 1 and 1 000 000?

P4. ⋆⋆ Trace Euclid's algorithm for $(18, 48)$ and for $(1071, 462)$.

P5. ⋆⋆ Compute the two check digits for the made-up first nine digits 2 0 2 6 1 0 0 1 5.

P6. ⋆⋆ **Find the bug.** A student wrote $avg \leftarrow g_1 + g_2 + g_3/3$ and tested it only with $0, 0, 0$. Explain why the test passed and give a test that exposes the bug.

P7. ⋆⋆ **Maximum of N numbers.** Read N and then N numbers one by one; print the largest. (Hint: keep a "largest so far".)

P8. ⋆⋆⋆ **Sum of digits.** Read a positive integer and print the sum of its digits (e.g. $4071 \to 12$). Use only mod, integer division and a loop.

P9. ⋆⋆⋆ **Convert.** Rewrite the Turkish-tea algorithm (Algorithm 1) using the pseudocode conventions of Section 3.8, with variables for the water temperature and the guest's preference.

References and further reading

[1] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed., Addison-Wesley, 1997, Section 1.1.

[2] G. Pólya, *How to Solve It*, Princeton University Press, 1945.

[3] C. Böhm and G. Jacopini, "Flow diagrams, Turing machines and languages with only two formation rules", *Communications of the ACM* 9(5), 366–371, 1966. https://doi.org/10.1145/355592.365646

[4] E. W. Dijkstra, "Notes on Structured Programming" (EWD249), 1970. https://www.cs.utexas.edu/~EWD/ewd02xx/EWD249.PDF

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022, Chapter 1.

[6] Microsoft, *C# documentation – tutorials for beginners*. https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/tutorials/

[7] Microsoft, *.NET download (.NET 10)*. https://dotnet.microsoft.com/en-us/download/dotnet/10.0